

Apunte de programación

Técnicas de Diseño de Algoritmos

20 de agosto de 2025

1. Introducción

Este apunte busca dar directivas generales para las prácticas de programación en la materia, sobre todo en el contexto de los talleres. No buscamos ser exhaustivos en los detalles de cada lenguaje, sino dar una guía para ayudarlos.

Qué asumimos Desde el principio de la materia, asumiremos que tienen una base sólida de programación imperativa, con práctica en Python (Introducción a la Programación, IP) y Java (Algoritmos y Estructura de Datos, AED), además de un conocimiento básico de testing. Durante la materia, les iremos enseñando nuevas técnicas para pensar algoritmos, pero la base de programación que tienen es suficiente para escribir el código necesario para los ejercicios. De hecho, en general les va a alcanzar con programar a nivel de funciones, sin necesidad de escribir clases nuevas (con algunas excepciones).

Tiempo de los algoritmos Dicho eso, hay algunas peculiaridades que en materias anteriores no se tuvieron en cuenta. Si bien en AED aprendieron sobre complejidad computacional, una teoría independiente a los lenguajes y detalles de implementación, en los talleres de esta materia tendremos requerimientos estrictos del tiempo real de ejecución de sus algoritmos. Esto es así para poder chequear automáticamente que sus algoritmos sean eficientes. Eso hace que algunos detalles de implementación empiecen a importar, siendo el más importante el lenguaje. Aunque los talleres tienen opción de ser entregados en muchos lenguajes diversos, **no damos garantías de que los tiempos se puedan conseguir con Python**. Esta es una restricción común en contextos donde los tiempos de cómputo importan.

Lenguajes recomendados Es por eso que desde la cátedra recomendamos implementar sus talleres en Java o, en caso de querer un desafío, en C++. Como verán, el salto de Java a C++ es relativamente chico (para lo que necesitarán en esta materia), porque el diseño de Java como lenguaje fue fuertemente influido por C++. Java resulta levemente más lento que C++ en tiempos de ejecución, aunque los talleres pueden ser aprobados en ambos lenguajes.

Relación entre complejidad y tiempo ¿Esto significa que debemos tirar a la basura lo que sabemos de complejidad? No, y en la materia le daremos bastante importancia al estudio de la complejidad de los algoritmos. Existe una relación bastante sólida entre complejidad y tiempo, aunque no es exacta por detalles de la máquina en la que ejecutemos y la constante que nos oculta la notación O . Para los talleres, deberemos calcular la complejidad de nuestro algoritmo e evaluarla en el valor máximo que pueden tomar las variables.

- Si nos da $\leq 10^8$ operaciones, estaremos bastante seguros de que nuestro algoritmo correrá en máximo 1 segundo.
- Si nos da entre 10^8 y 10^9 operaciones, puede que el algoritmo entre en tiempo pero dependerá de constantes.
- Si nos da $> 10^9$, lo más probable es que nuestro algoritmo se exceda de tiempo.

Agradecimientos Este documento toma material y se inspira en las diapositivas de “Introducción a la programación competitiva” de Agustín Santiago Gutiérrez, dictadas durante varios *Training Camp*, accesibles en https://www.pc-arg.com/media/attachment/ipc_gnsCMx2.pdf.

2. Código base y Entrada-Salida

A continuación daremos un código base para cada lenguaje (con las declaraciones requeridas) y hablaremos de como recibir parámetros y devolver resultados. La entrada y salida en el juez que usaremos se hace mediante el mecanismo estándar, que es la consola.

2.1. Python

En Python, no necesitan una estructura particular de código. Para leer un dato de la consola, pueden utilizar `input()`. Esta función nos dará el próximo string que se ingrese en la consola. Si deseamos leer otro tipo de dato (como `int` o `float`), deberemos leer el string y cambiarle el tipo. Por ej: `n = int(input())`. Para la salida de datos, pueden usar `print(<DATA1>, <DATA2>, ..., <DATAN>)`, que imprime las variables/strings que le ingresen por parámetro separados por espacios y termina la línea con un *enter*.

2.2. Java

Para trabajar en Java, necesitarán una clase (que puede llamarse como quieran, pero debe llamarse igual que el archivo) que tenga un método `main`:

```
1 public class Main {
2     public static void main(String[] args) {
3
4     }
```

```
5 }
```

En Java hay varios métodos para manejar entrada y salida por consola, pero recomendamos usar `BufferedReader+Scanner` y `BufferedWriter+PrintWriter` para que se realice eficientemente. Estos deben ser importados de `java.io` y `java.util`.

```
1 import java.io.*;
2 import java.util.*;
3
4 class Main {
5     public static void main(String [] args) {
6         baz;
7         BufferedReader br = new BufferedReader(
8             new InputStreamReader(System.in));
9         BufferedWriter bw = new BufferedWriter(
10            new OutputStreamWriter(System.out));
11         Scanner scanner = new Scanner(br);
12         PrintWriter printer = new PrintWriter(bw);
13         int n = scanner.nextInt();
14         long total = 0;
15         for (int i = 0; i < n; i++) {
16             long x = scanner.nextLong();
17             total += x;
18             printer.println(total);
19         }
20         printer.close();
21     }
22 }
```

Notar que `printer` es un objeto de clase `PrintWriter`¹, que tiene un método `println` para imprimir una línea de texto (terminada en *enter*) y `scanner` es un objeto de clase `Scanner`² con métodos `nextInt`, `nextLong`, `nextLine` (leer una línea entera como string), etc.

Habrán notado que el ejemplo usa el tipo `long`. Un `long` es un número entero, como `int`, pero con mayor cantidad de bits de almacenamiento, por lo que puede trabajar con números más grandes.

2.3. C++

En C++, el código básico requiere que haya una función `main` que devuelva un `int`. Ese `int` suele ser 0 si se ejecutó el programa correctamente, y otro número si ocurrió un error. En la materia no nos va a importar esto, por lo que podemos simplemente siempre hacer `return 0;`.

```
1 int main() {
2     return 0;
3 }
```

Para trabajar con entrada/salida, usaremos `cin` y `cout` del paquete estándar `iostream`.

¹Documentación oficial de `PrintWriter` de Java.

²Documentación oficial de `Scanner` de Java.

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(NULL);
7     int x;
8     int y;
9     cin >> x >> y;
10    cout << x+y << "\n";
11 }

```

La línea 9 `cin >> x >> y;` lee dos valores de la consola (separados por espacio o *enter*) y los almacena en las variables `x` e `y`, teniendo en cuenta el tipo de esas variables. Luego, la línea 10 `cout << x+y << "\n";` imprime su suma por consola y termina con el string `\n` que representa un *enter*.

3. Estructuras de la biblioteca estándar

Las siguientes estructuras, que vieron en AED, se encuentran en la biblioteca estándar tanto de Java como de C++ y pueden ser usadas libremente. Dejamos enlaces a la documentación de cada una.

Estructura	En Java	En C++
Arreglo redimensionable	ArrayList	vector
Lista doblemente enlazada	LinkedList	list
Conjunto sobre árbol balanceado	TreeSet	set
Diccionario sobre árbol balanceado	TreeMap	map
Pila	Stack	stack
Cola	Queue	queue
Cola de prioridad (heap)	PriorityQueue	priority_queue

Las estructuras en Java deben importarse de `java.util` (ejemplo: `import java.util.LinkedList;`). En C++, cada estructura tiene su propia biblioteca de `include` (ejemplo: `#include <list>`) excepto por `priority_queue` que se importa de `#include <queue>`.

4. Apunte básico de C++

4.1. Iterando estructuras

```
1 // For loop
2 for(int i = 0; i < estructura.size();i++) {
3     estructura[i] = ...
4 }
5
6 // For range loop
7 for(type t: estructura) {...} // Se copia cada valor
8 for(type& t: estructura) {...} // Se pasa por referencia
```

4.1.1. Iteradores

Un iterador es un objeto que actúa como un puntero que permite recorrer los elementos de una colección de manera eficiente. Podemos avanzar, retroceder y acceder al valor de la posición a través de ellos. Al ser puntero, para acceder al valor de la posición debemos usar el `*`.

```
1     for(auto it = estructura.begin(); it != estructura.end(); it++) {
2         cout << *it << endl;
3     }
```

4.2. Secuencias

Este tipo de estructura tiene varias implementaciones. Lo que ustedes conocen en python como *list*

```
1     enteros = [1,1,2,3,5,8,13]
```

4.2.1. Vectores

Documentación oficial

```
1     #include <vector>
```

Tener en cuenta que por como funcionan los vectores en C++, estos deben pedir memoria para almacenar los elementos, una vez que se queda sin espacio piden el doble de memoria. Con lo cual, las primeras inserciones son más caras que pedir memoria no es una operación "gratuita". Esto lo podemos evitar si sabemos cuántos elementos va a tener usando los otros constructores.

```
1     // Constructores
2     vector<int> ceros(3, 0);    // [0,0,0]
3     vector<int> unos = {1,1,1}; // A partir de C++17
4
5     vector<int> fibo;           // empty
6     fibo.assign(3,1);           // [1,1,1]
7     fibo.resize(5);             // Aumentamos la capacidad a 5 (size = 3)
8
9     // Insertar y acceder
```

```

10     fibo.push_back(3); // [1,1,1,3]
11
12     fibo[2] = 2;       // [1,1,2,3]
13     fibo[4] = 5;       // Segmentation Fault
14
15     fibo.back();        // 3
16     fibo.front();       // 1
17
18     // Longitud
19     fibo.size() > 3
20
21     // Eliminar
22     fibo.pop_back();    // [1,1,2]
23     fibo.clear();       // []

```

4.2.2. Lista Enlazada

Documentación oficial

```
1     #include <list>
```

Tener en cuenta que acceder a la posición i cuesta $O(i)$. Al ser una lista doblemente enlazada, las inserciones (tanto al principio como al final de la lista) se realizan en $O(1)$. Por otro lado, no cuentan con el famoso acceder con `[]` con lo cual debemos usar iteradores.

```

1     // Constructores
2     list<int> ceros(2, 0); // [0,0]
3     list<int> fibo; // empty
4     fibo.assign(3,1);
5
6     // Acceso
7     // Iteradores
8     // Iterador que apunta al principio
9     list<int>::iterator it = fibo.begin();
10
11     // Podemos usar auto que infiere el tipo de dato
12     auto end_it = fibo.end();
13
14     // Avanzar el iterador
15     it++; // avanza al siguiente
16     next(it, n); // avanza n pasos
17
18     // Iterando
19     for (auto it = fibo.begin(); it != fibo.end(); it++) {
20         cout << "Valor:_" << *it << endl;
21     }
22
23     // Insertar
24     fibo.push_back(3); // [1,1,1,3]
25     fibo.push_front(0); // [0,1,1,1,3]
26     fibo.insert(next(fibo.begin(), 3), 2); // [0,1,1,1,2,3]
27
28     fibo.back(); // 3
29     fibo.front(); // 0
30

```

```

31 // Longitud
32 fibo.size()
33
34 // Eliminar
35 fibo.pop_back(); // [0,1,1,1,2]
36 fibo.pop_front(); // [1,1,1,2]
37 fibo.erase(fibo.begin()); // [1,1,2]
38 fibo.clear(); // []

```

4.2.3. Arreglos

Documentación oficial

```
1 // No necesitan include
```

Tienen tamaño fijo. Crear un arreglo de tamaño n cuesta $O(1)$, pero inicialmente pueden tener cualquier valor que había en las posiciones de memoria tomadas (“se inicializan con basura”).

```

1 // Constructores
2 int fibo[3]; // [125665304, 38438493, 34]
3 memset(fibo, 1, sizeof(fibo)); // [1,1,1]
4
5 // Insertar y acceder
6 fibo[2] = 2; // [1,1,2]
7 fibo[4] = 5; // Segmentation Fault

```

4.3. Algoritmos útiles

Documentación oficial

```
1 #include <algorithm>
```

La biblioteca estándar `algorithm` cuenta con implementaciones de muchos algoritmos útiles.

4.3.1. Ordenamiento (sort)

La función `sort` toma como parámetros un iterador al principio y otro al final de la secuencia. También sirve para ordenar solo una parte de la secuencia, pasando los iteradores correspondientes a ese intervalo.

```

1 vector<int> v = {5, 3, 2, 1, 4};
2 sort(v.begin(), v.end()); // [1, 2, 3, 4, 5]
3
4 int arr[5] = {5, 3, 2, 1, 4};
5 sort(arr, arr + 5); // [1, 2, 3, 4, 5]

```

4.3.2. Búsqueda binaria

La función `binary_search` toma como parámetros un iterador al principio y otro al final de la secuencia, además del valor buscado.

```

1  vector<int> v = {5, 3, 1, 4};
2  sort(v.begin(), v.end()); // [1, 3, 4, 5]
3
4  binary_search(v.begin(), v.end(), 3); // true
5  binary_search(v.begin(), v.end(), 2); // false

```

Además, existen las funciones `lower_bound` (que devuelve un iterador al primer elemento que no es menor al buscado) y `upper_bound` (que devuelve un iterador al primer elemento mayor al buscado). Sirven para buscar un elemento, pero también para encontrar y recorrer un rango de valores.

```

1  vector<int> v = {5, 3, 2, 1, 4};
2  sort(v.begin(), v.end()); // [1, 2, 3, 4, 5]
3
4      auto lb = lower_bound(v.begin(), v.end(), 2);
5      auto up = upper_bound(v.begin(), v.end(), 4);
6
7  // Imprime los numeros 2 y 3
8  // Es decir, los que estan en el rango [2, 4)
9  for (auto it = lb; it != up; it++) {
10      cout << *it << endl;
11  }

```

4.4. Diccionarios

Lo que ustedes conocen en python como *dict*.

```
1 materias = {1:"TDA", 2:"PLP", 3:"SD"}
```

Este tipo de estructura tiene 2 implementaciones.

4.4.1. Map

Documentación oficial

```
1 #include <map>
```

No es un diccionario común, ya que mantiene las claves ordenadas a través de un árbol balanceado (*Red-Black Tree*). Con lo cual, tanto la búsqueda como la inserción son $O(\log n)$.

El operador `[e]` genera un elemento vacío con clave `e` en caso de que no esté definido, si no devuelve lo que se encuentre almacenado.

```

1  // Constructores
2  map<int, int> fibo = { {5,5} , {6,8}, {7,13} };
3
4  map<int, string> materias; // empty
5
6  // Insertar y acceder
7  materias.insert(1, "TDA"); // {1:"TDA"}
8  materias.insert(3, "SD"); // {1:"TDA", 3:"SD"}
9  materias.insert(2, "PLP"); // {1:"TDA", 2:"PLP", 3:"SD"}
10 materias[1] = "TDA";
11
12 materias.at(1); // "TDA"

```



```

13     materias[1];                // "TDA"
14     materias.at(5);             // Exception
15     materias[5];                // ""
16
17     // Iterando
18     for (auto it = materias.begin(); it != materias.end(); ++it) {
19         cout << "Clave:_" << it->first << endl;
20         cout << "Valor:_" << it->second << endl;
21     }
22
23     // Otros
24     materias.count(1);           // 1
25     materias.count(5);           // 0
26
27     // Longitud
28     materias.size();
29
30     // Eliminar
31     materias.erase(materias.begin()); // {3:"SD"}
32     materias.erase(3);             // {2:"PLP"}
33     materias.clear();              // {}

```

4.4.2. Unordered Map

Documentación oficial

```
1     #include <unordered_map>
```

Tiene las mismas operaciones que el `map` pero no mantiene las claves ordenadas ya que está implementado con tabla de hash.

4.5. Conjuntos

Este tipo de estructura tiene 2 implementaciones al estilo de los diccionarios

4.5.1. Set

Documentación oficial

Análogo al `map`, mantiene la estructura ordenada usando un árbol balanceado.

```

1     #include <set>
2
3     // Constructores
4     set<int> fibo = {1,1,2,3,5,8};
5     set<int> vacio;
6
7     // Insertar y acceder
8     fibo.insert(13);           // {1,1,2,3,5,8,13}
9
10    auto it = fibo.find(8)
11
12    // Iterando
13    for (auto it = fibo.begin(); it != fibo.end(); ++it) {
14        cout << "Valor:_" << *it << endl;

```

```

15     }
16
17     // Otros
18     fibo.count(1);           // 1
19     fibo.count(4);           // 0
20     fibo.upper_bound();       // iterador al max
21     fibo.lower_bound();       // iterador al min
22
23     // Longitud
24     fibo.size();
25
26     // Eliminar
27     fibo.erase(fibo.begin()); // {1,2,3,5,8,13};
28     fibo.erase(3);            // {1,2,5,8,13};
29     fibo.clear();              // {}

```

4.5.2. Unordered Set

Documentación oficial

```
1 #include <unordered_set>
```

Similar al *set* pero implementado con tabla de hash

4.6. Filas

Documentación oficial

```
1 #include <stack>
```

Una implementación del tipo de fila LIFO (*last in-first out*, los elementos se eliminan en el orden opuesto al que se ingresan).

```

1     // Constructores
2     stack<int> fila;
3
4     // Insertar y acceder
5     fila.push(16);           // [16]
6     fila.push(2);            // [2, 16]
7     fila.push(26);           // [26, 2, 16]
8
9     // Otros
10    fila.top();               // 26
11
12    // Longitud
13    fila.size();              // 3
14
15    // Eliminar
16    fila.pop();               // [2, 16]
17
18    // Iterando
19    while(not fila.empty()) {
20        int n = fila.top();
21        fila.pop();
22        ...
23    }

```

4.7. Colas

4.7.1. Queue

Documentación oficial

```
1  #include <queue>
```

Una implementación del tipo de cola FIFO (*first in-first out*, los elementos se eliminan en el mismo orden que se ingresan).

```
1  // Constructores
2  queue<int> cola;
3
4  // Insertar y acceder
5  cola.push(16);           // [16]
6  cola.push(2);           // [16, 2]
7  cola.push(26);          // [16, 2, 26]
8
9  // Otros
10 cola.top();              // 16
11 cola.front();           // 16
12 cola.back();            // 26
13
14 // Longitud
15 cola.size();            // 3
16
17 // Eliminar
18 cola.pop();             // [2, 26]
19
20 // Iterando
21 while(not cola.empty()) {
22     int n = cola.top();
23     cola.pop();
24     ...
25 }
```

4.7.2. Cola de prioridad (Max-Heap)

Documentación oficial

```
1  #include <queue>
```

Tiene las mismas funciones que la `queue` normal, excepto `front` y `back`. Para ver el primer elemento, se usa el método `top`. Además, su comportamiento no es FIFO, ordena por máxima prioridad. Si se quiere usar como *Min-Heap* tenemos que ingresar los números en negativo.

4.8. Ejemplos de uso

```
1  #include <vector>
2  #include <list>
3  #include <queue>
4  #include <unordered_set>
5  #include <set>
6  #include <map>
7  #include <unordered_map>
8
9  // La mayor cantidad de elementos es 1000
10 const int MAXN = 1000;
11
12 int mi_arreglo[MAXN];
13 vector<int> mi_vector;
14 list<int> mi_lista;
15
16 unordered_map<int, int> mi_umap;
17 map<int, int> mi_map;
18
19 unordered_set<int> mi_uset;
20 set<int> mi_set;
21
22 queue<int> mi_queue;
23 priority_queue<int> mi_heap;
24
25 // Tenemos n items y queremos encontrar e
26 bool encontrar_elemento(int n, int e) {
27     for(int i = 0; i < n; i++) {
28         if (mi_arreglo[i] == e) return true;
29         if (mi_vector[i] == e) return true;
30     }
31
32     for(auto it = mi_lista.begin(); it != mi_lista.end(); it++) {
33         if (*it == e) return true;
34     }
35
36     // O(1)
37     if (mi_umap.count(e) != 0) return true;
38
39     // O(log n)
40     if (mi_map.count(e) != 0) return true;
41
42     // O(1)
43     if (mi_uset.count(e) != 0) return true;
44
45     // O(log n)
46     if (mi_set.count(e) != 0) return true;
47
48     // O(n)
49     queue<int> copia;
50     bool found = false;
51     while(not mi_queue.empty()) {
52         int item = mi_queue.top();
53         if (item == e) found = true;
54         copia.push(item);
55         mi_queue.pop();
```

```

56     }
57     mi_queue = copia;
58     if (found) return true;
59
60     // O(n log n)
61     priority_queue<int> copia_heap;
62     bool found = false;
63     while(not mi_heap.empty()) {
64         int item = mi_queue.top(); // O(1)
65         if (item == e) found = true;
66         copia_heap.push(item);      // O(log n)
67         mi_queue.pop();             // O(log n)
68     }
69     mi_queue = copia;
70     if (found) return true;
71 }
72
73 int main() {
74     ios::sync_with_stdio(false);
75     cin.tie(NULL);
76
77     // Suponemos que el input es un entero n < MAXN
78     // y luego n enteros
79
80     int n;
81     cin >> n;
82
83     // Leemos el input
84     for(int i = 0; i < n; i++) {
85         int x; cin >> x;
86
87         mi_vector.push_back(x);
88         mi_arreglo[n] = x;
89         mi_lista.push_back(x);
90
91         mi_map[x] = 1;
92         mi_umap[x] = 1;
93
94         mi_set.insert(x);
95         mi_uset.insert(x);
96
97         mi_queue.push(x);
98         mi_heap.push(x);
99     }
100
101     // Buscamos el 23
102     bool contiene = encontrar_elementos(23);
103
104     // Preparamos la respuesta con un inline if
105     string respuesta = contiene ? "SI" : "NO";
106
107     // Imprimimos la respuesta
108     cout << respuesta << "\n";
109
110     return 0;
111 }

```